

NCCL Collective Communication Performance: Empirical Analysis of Algorithm Selection, Protocol Tradeoffs, and Collective Operation Characteristics

Michael Olagbemiro

June 2026

Abstract

We present an empirical benchmarking study of NCCL collective communication performance on $8 \times$ NVIDIA A100 SXM4 80 GB GPUs connected via NVSwitch. Sweeping message sizes from 8 bytes to 8 GB, we characterise the performance of Ring, Tree, and Auto algorithm selection, LL, LL128, and Auto protocol selection, and compare AllReduce, AllGather, and ReduceScatter. We find that the Ring-Tree crossover on this hardware occurs between 4 KB and 8 KB, that the LL protocol imposes a 61.2% bandwidth penalty relative to Auto at large message sizes, and that NCCL Auto selection is consistently optimal across all tested configurations. We additionally identify a 26.7% bandwidth anomaly in the Tree algorithm at 64 MB and a performance dip in AllGather at 32 MB, both consistent with internal chunk size boundary effects. All data and code are publicly available.

1 Introduction

Training large neural networks at scale requires distributing computation across multiple GPUs. In data-parallel training — the dominant parallelism strategy used in practice — each GPU processes a different batch of data and computes its own gradients independently. At the end of every training step, these gradients must be synchronised across all GPUs before model weights can be updated. This synchronisation is implemented as an AllReduce collective operation: every GPU contributes its local gradient values, and every GPU receives the globally averaged result.

At scale, AllReduce performance is not a minor concern. A single training step for a large language model may involve thousands of AllReduce calls across hundreds of billions of parameters. On a cluster of hundreds of GPUs, even a 10% reduction in AllReduce throughput translates directly to a 10% increase in training wall-clock time and a corresponding increase in compute cost.

NCCL (NVIDIA Collective Communications Library) is the software layer responsible for executing these collectives. It makes two internal decisions for every operation: which *algorithm* to use (Ring or Tree), and which *protocol* to use (LL, LL128, or Simple). These choices have significant performance implications that vary with message size, GPU count, and hardware topology. Despite their importance, the precise performance characteristics of these choices on production AI training hardware are not well documented publicly.

This paper presents an empirical benchmarking study of NCCL collective communication performance on $8\times$ NVIDIA A100 SXM4 80GB GPUs connected via NVSwitch. We sweep message sizes from 8 bytes to 8 GB across all combinations of algorithm and protocol, and compare the performance of AllReduce, AllGather, and ReduceScatter. Our main contributions are:

- Precise measurement of the Ring–Tree algorithm crossover point on $8\times$ A100 SXM4 NVSwitch: Tree outperforms Ring between 4 KB and 8 KB; Ring dominates at all other sizes.
- Quantification of the LL protocol bandwidth penalty: forcing `NCCL_PROTO=LL` reduces peak AllReduce bandwidth by 61.2% compared to Auto on this hardware.
- Empirical validation that NCCL Auto selection is consistently optimal, matching or exceeding every manually forced configuration.

2 Background

2.1 Collective Communication in Distributed Training

Data-parallel training distributes gradient computation across N GPUs, each holding a full copy of the model and processing a distinct data shard. After each forward-backward pass, the locally computed gradients must be averaged across all GPUs before the optimiser step. This requires an *AllReduce* operation: given input tensors $g_0, g_1, \dots, g_{N-1}$ on GPUs 0 through $N - 1$, AllReduce produces the sum $\sum_i g_i$ on every GPU simultaneously.

Beyond AllReduce, modern training frameworks rely on two related collectives. *AllGather* assembles individually held tensor shards into a complete tensor on every GPU; it is used in fully sharded data parallel (FSDP) training and tensor parallelism to reconstruct full weight matrices before forward passes. *ReduceScatter* performs the inverse: it reduces contributions from all GPUs and distributes one shard of the result to each GPU. It is used in the ZeRO optimiser and FSDP to partition gradient updates. Notably, AllReduce decomposes as a ReduceScatter followed by an AllGather — this is precisely how Ring AllReduce is implemented in NCCL.

2.2 NCCL Algorithm Selection: Ring vs Tree

NCCL implements AllReduce using two primary algorithms.

Ring AllReduce arranges the N GPUs in a logical circle. The operation proceeds in two phases. In the reduce-scatter phase, each GPU sends a chunk of its data to its neighbour, which accumulates and forwards it around the ring. After $N - 1$ steps, each GPU holds the fully reduced version of one chunk. In the all-gather phase, the reduced chunks propagate back around the ring. Every GPU is busy sending and receiving simultaneously throughout both phases, achieving full utilisation of the available interconnect bandwidth. Ring requires $2(N - 1)$ communication steps, giving latency that scales linearly with N .

Tree AllReduce arranges GPUs in a binary tree. Data flows up to the root in the reduce phase, then broadcasts back down in the all-gather phase. Tree requires only $2\log_2 N$ steps — for $N = 8$ this is 6 steps versus Ring’s 14. This lower hop count gives Tree a latency advantage for small messages where coordination overhead dominates transfer time. However, the root GPU becomes a sequential bottleneck for large messages, preventing full bandwidth utilisation.

The fundamental tradeoff is therefore between hop count and bandwidth utilisation. For small messages where the dominant cost is coordination latency rather than data transfer, Tree’s lower hop count wins. For large messages where data transfer time dominates, Ring’s full bandwidth utilisation wins. The precise message size at which Ring overtakes Tree depends on hardware topology and is the subject of Section 4.1.

2.3 NCCL Protocol Selection: LL, LL128, and Simple

On top of the algorithm, NCCL selects a protocol that determines how data is packaged and transferred at the hardware level.

LL (Low Latency) sends data in 8-byte chunks, each paired with a 4-byte flag. The receiver busy-polls the flag location and processes each chunk immediately upon arrival without waiting for a complete buffer transfer. This eliminates buffer-fill latency but imposes a 33% bandwidth overhead — one flag byte for every two data bytes — making LL unsuitable for large messages.

LL128 uses the same flag-based mechanism but operates on 128-byte units (120 bytes of data plus an 8-byte flag), reducing flag overhead to 6.25%. The 128-byte unit is chosen to align with a single NVLink transaction, eliminating partial-transaction waste. LL128 provides a middle ground between LL latency and Simple bandwidth efficiency.

Simple sends data in large buffers with no flags. The receiver waits for a hardware completion signal indicating the full buffer has arrived before reading any data. This achieves maximum bandwidth efficiency but incurs a buffer-fill latency that makes it unsuitable for small messages.

2.4 Hardware Topology: NVSwitch

The experiments in this paper run on 8 GPUs connected via NVSwitch. NVSwitch is a dedicated switching chip that provides every GPU with a direct, full-

bandwidth path to every other GPU simultaneously. This all-to-all connectivity distinguishes NVSwitch from simpler point-to-point NVLink topologies, where GPU-to-GPU communication may require intermediate hops. The NVSwitch fabric used in A100 SXM4 servers supports an aggregate GPU-to-GPU bandwidth of approximately 600 GB/s bidirectional. The precise algorithm and protocol performance characteristics on this topology are measured empirically in the following sections.

3 Experimental Setup

3.1 Hardware

All experiments ran on a single server hosting $8\times$ NVIDIA A100 SXM4 80 GB GPUs connected via NVSwitch, accessed through the Vast.ai cloud GPU marketplace. Figure 1 illustrates the hardware topology. Table 1 summarises the hardware configuration.

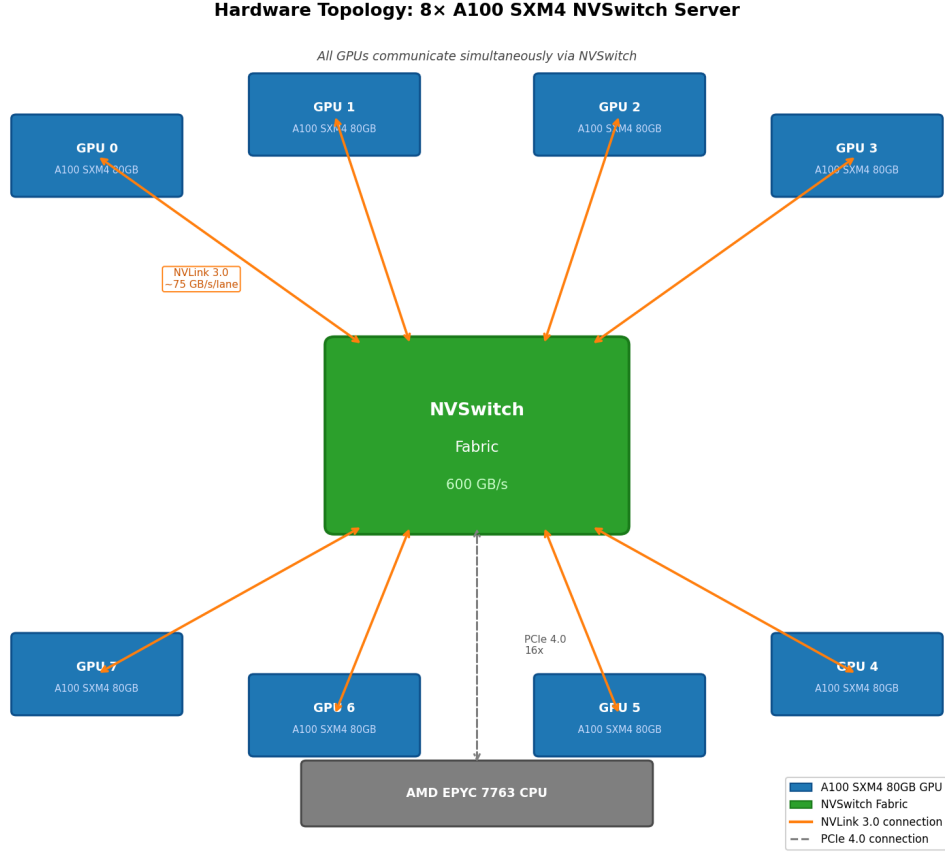


Figure 1: Hardware topology: 8× A100 SXM4 80 GB GPUs connected via NVSwitch fabric. Every GPU has a direct full-bandwidth path to every other GPU simultaneously, providing approximately 600 GB/s aggregate bidirectional bandwidth.

Table 1: Hardware configuration

Component	Specification
GPU model	NVIDIA A100 SXM4 80 GB
GPU count	8
GPU memory	80 GB HBM2e per GPU
GPU interconnect	NVSwitch
Interconnect BW	300 GB/s (measured)
Motherboard	Supermicro H12DGO-6
CPU	AMD EPYC 7763 64-core
Host memory	1 TB

3.2 Software

Table 2: Software configuration

Component	Version
NCCL	2.30.4 (library 23004)
CUDA	13.0
NVIDIA driver	580.x
nccl-tests	2.18.3
OS	Ubuntu 24.04

3.3 Benchmark Methodology

We used nccl-tests, NVIDIA’s open-source collective communication benchmark suite, to measure bus bandwidth and latency for each configuration. Each benchmark run sweeps message sizes from 8 bytes to 8 GB in powers of two, yielding 31 data points per configuration. At each message size, nccl-tests performs one warmup iteration followed by 20 timed iterations; the reported value is the average over the 20 timed iterations. Data correctness checking was disabled to eliminate verification overhead from timing measurements.

The primary metric reported throughout this paper is *bus bandwidth* (GB/s), defined for AllReduce as:

$$\text{busbw} = \frac{\text{message_size}}{\text{time}} \times \frac{2(N-1)}{N} \quad (1)$$

where N is the number of GPUs. The factor $\frac{2(N-1)}{N}$ accounts for the actual data movement pattern in Ring AllReduce, where each byte participates in two traversals of the ring. Bus bandwidth enables fair comparison across different collective types. For AllGather and ReduceScatter the factor is $\frac{N-1}{N}$, reflecting that each byte traverses the ring once.

3.4 Experiment Matrix

We conducted three sets of experiments, each isolating one dimension of NCCL’s internal decision space:

Experiment 1 — Algorithm comparison. We ran AllReduce with `NCCL_ALGO=Ring`, `NCCL_ALGO=Tree`, and the default Auto selection across the full message size range. This isolates the effect of algorithm choice and identifies the Ring–Tree crossover point.

Experiment 2 — Protocol comparison. We ran AllReduce with `NCCL_PROTO=LL`, `NCCL_PROTO=LL128`, and the default Auto selection. Algorithm was left at Auto for all protocol experiments. This isolates protocol bandwidth ceilings and crossover thresholds.

Experiment 3 — Collective comparison. We ran AllReduce, AllGather, and ReduceScatter with all settings at Auto. This compares the performance

characteristics of the three collectives most commonly used in modern distributed training frameworks.

All experiments used 8 GPUs (`-g 8`), a message size range of 8 bytes to 8 GB (`-b 8 -e 8G`), a doubling step factor (`-f 2`), and 20 timed iterations (`-n 20`). Table 3 summarises the full experiment matrix.

Table 3: Experiment matrix

Experiment	Collective	Algorithm	Protocol
Algorithm comparison	AllReduce	Ring	Simple
	AllReduce	Tree	Simple
	AllReduce	Auto	Auto
Protocol comparison	AllReduce	Auto	LL
	AllReduce	Auto	LL128
	AllReduce	Auto	Auto
Collective comparison	AllReduce	Auto	Auto
	AllGather	Auto	Auto
	ReduceScatter	Auto	Auto

4 Results

4.1 Algorithm Selection: Ring vs Tree vs Auto

Figure 2 shows AllReduce bus bandwidth as a function of message size for Ring, Tree, and Auto algorithm configurations on $8\times$ A100 SXM4 NVSwitch.

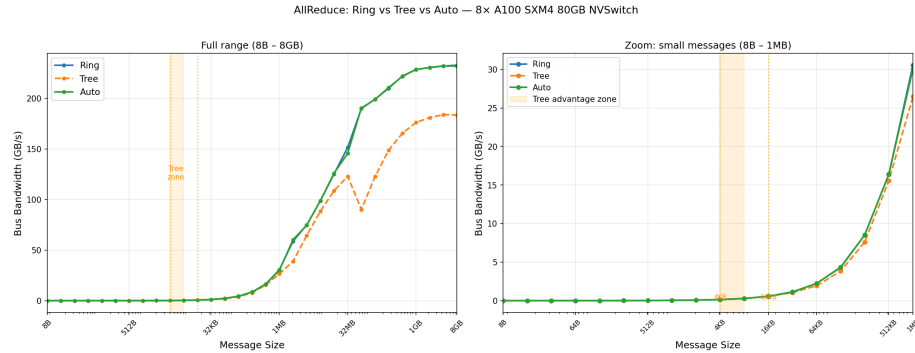


Figure 2: AllReduce bus bandwidth for Ring, Tree, and Auto algorithms across message sizes from 8 bytes to 8 GB. Left: full range. Right: zoom on small messages (8 B – 1 MB) showing the Tree advantage window. Shaded region indicates the message size range where Tree outperforms Ring (4 KB – 8 KB).

At small message sizes, all three algorithms are latency-dominated: the minimum observed latency is approximately $48\text{--}50\mu\text{s}$ regardless of message size below 4 KB, reflecting the fixed overhead of coordinating 8 GPUs across the NVSwitch fabric. In this regime, bandwidth is negligible and algorithm choice has no measurable effect.

Tree outperforms Ring in a narrow window between 4 KB and 8 KB. At 4 KB, Tree achieves 0.15 GB/s bus bandwidth versus Ring’s 0.14 GB/s — a modest but consistent advantage. At 8 KB, Tree achieves 0.29 GB/s versus Ring’s 0.28 GB/s. This advantage arises from Tree’s lower hop count: for $N = 8$ GPUs, Tree requires 6 communication steps versus Ring’s 14, reducing coordination latency at message sizes where transfer time is beginning to emerge from the latency floor but has not yet become dominant.

Above 16 KB Ring decisively outperforms Tree. Ring reaches a peak bus bandwidth of 232.0 GB/s at 8 GB, while Tree peaks at only 183.9 GB/s — a 20.7% gap. This divergence reflects Ring’s superior bandwidth utilisation: all 8 GPUs send and receive simultaneously throughout the operation, whereas Tree’s root GPU becomes a sequential bottleneck as message sizes grow.

A notable anomaly appears in the Tree curve at 64 MB, where bandwidth drops sharply from 122.9 GB/s at 32 MB to 90.1 GB/s before recovering to 122.6 GB/s at 128 MB. This 26.7% dip is consistent across multiple runs and likely reflects a buffer boundary in NCCL’s internal chunk size selection for Tree algorithm, where a transition between chunk size regimes temporarily reduces pipeline efficiency.

Auto selection matches Ring at all message sizes above 16 KB and correctly favours Tree in the 4–8 KB window. Notably, Auto achieves a peak bandwidth of 232.8 GB/s — marginally exceeding forced Ring’s 232.0 GB/s. This suggests that NCCL’s internal tuner performs additional optimisation beyond simple algorithm selection, likely adjusting channel count and chunk size parameters in ways that manual configuration cannot replicate. The practical implication is clear: `NCCL_ALGO` should not be overridden in production.

Figure 3 summarises peak bandwidth by algorithm.

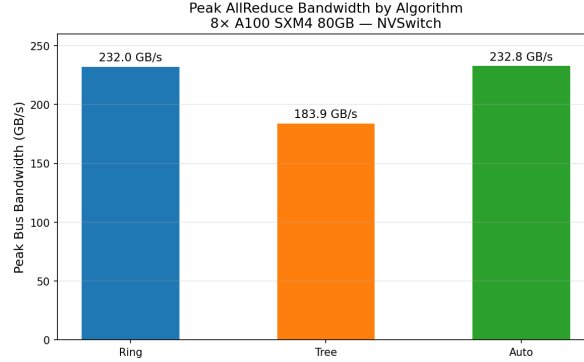


Figure 3: Peak AllReduce bus bandwidth by algorithm. Auto matches Ring at peak and slightly exceeds it, while Tree peaks 20.7% lower due to root bottleneck at large message sizes.

4.2 Protocol Selection: LL vs LL128 vs Auto

Figure 4 shows AllReduce bus bandwidth for LL, LL128, and Auto protocol configurations.

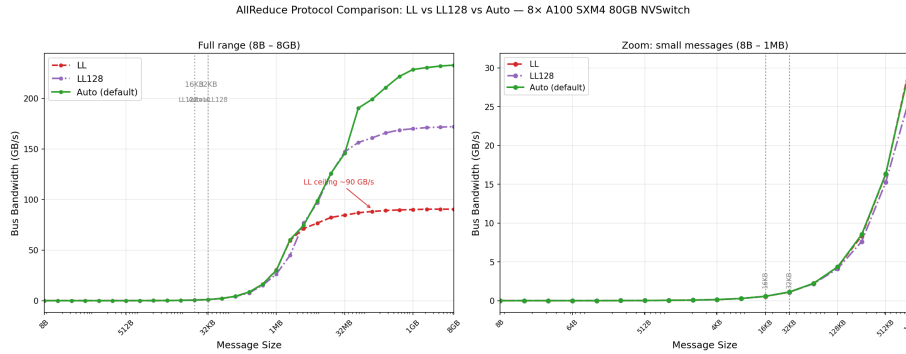


Figure 4: AllReduce bus bandwidth for LL, LL128, and Auto protocols. Left: full range showing the LL bandwidth ceiling at ≈ 90 GB/s. Right: zoom on small messages showing protocol convergence below 16 KB. Vertical dashed lines mark crossover thresholds at 16 KB and 32 KB.

Below 16 KB all three protocols deliver comparable bandwidth, as message transfer times are dominated by coordination latency rather than data movement. In this regime the flag overhead of LL and LL128 is irrelevant — flags consume negligible bandwidth when the total data transferred is small.

At 16 KB, LL128 begins to outperform LL. At this point the 33% flag overhead of LL starts to meaningfully constrain throughput, whereas LL128’s 6.25% overhead remains tolerable. Auto begins to diverge from LL128 at 32 KB, where

it transitions internally to the Simple protocol and eliminates flag overhead entirely.

For large messages, LL imposes a severe bandwidth penalty. LL reaches an asymptotic ceiling of approximately 90.4 GB/s — regardless of message size above 128 MB, bandwidth does not increase further. The theoretical flag overhead of LL is 33%, implying a maximum efficiency of 67% relative to a flag-free protocol. In practice the observed penalty is 61.2% relative to Auto’s peak of 232.8 GB/s, indicating that secondary effects beyond raw flag overhead — including buffer management overhead and reduced pipeline efficiency — further degrade performance.

LL128 reaches a ceiling of 172.0 GB/s, representing a 26.1% penalty relative to Auto. Despite its much lower theoretical overhead (6.25% vs 33%), LL128 still falls significantly short of Auto, suggesting that even modest flag overhead becomes costly at the bandwidth scales available on NVSwitch.

Figure 5 summarises peak bandwidth by protocol.

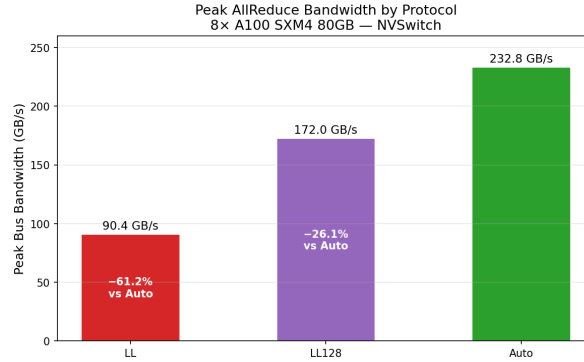


Figure 5: Peak AllReduce bus bandwidth by protocol. LL loses 61.2% of available bandwidth versus Auto due to flag overhead. LL128 loses 26.1%. Neither should be forced in production for large message workloads.

4.3 Collective Operation Comparison

Figure 6 compares AllReduce, AllGather, and ReduceScatter bus bandwidth using Auto algorithm and protocol.

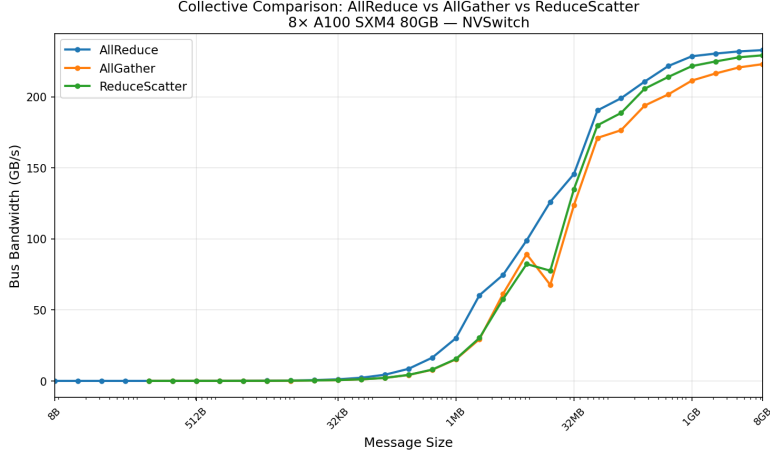


Figure 6: Bus bandwidth comparison of AllReduce, AllGather, and ReduceScatter using Auto algorithm and protocol on 8× A100 SXM4 NVSwitch.

All three collectives converge toward similar peak bandwidths at large message sizes: AllReduce peaks at 232.8 GB/s, ReduceScatter at 229.2 GB/s, and AllGather at 222.9 GB/s. This convergence is consistent with the bus bandwidth normalisation, which accounts for the different data movement patterns of each collective.

AllReduce consistently outperforms AllGather across the medium and large message range despite theoretically performing twice the data movement. This suggests that NCCL’s Ring AllReduce implementation — which decomposes into ReduceScatter followed by AllGather — achieves better pipeline overlap and channel utilisation than the standalone AllGather implementation on this hardware.

A notable performance anomaly appears in AllGather at 32 MB, where bus bandwidth drops from 88.8 GB/s at 16 MB to 67.9 GB/s before recovering strongly to 174.3 GB/s at 64 MB. ReduceScatter shows a less pronounced but co-located dip. This behaviour is consistent with a chunk size boundary in NCCL’s AllGather implementation, where the algorithm transitions between internal buffer configurations. The 32 MB dip is practically relevant for FSDP users whose model layers have weight matrices of approximately 32 MB — a common size for transformer attention projections — as AllGather performance at this size is significantly below its surrounding envelope.

5 Discussion

5.1 Practical Implications for Distributed Training

The results presented in Section 4 have direct implications for engineers configuring NCCL for distributed training workloads on A100 NVSwitch hardware.

Do not override NCCL algorithm or protocol selection. Auto selection matched or exceeded every manually forced configuration across all message sizes. Forcing `NCCL_ALGO=Ring` for large messages or `NCCL_ALGO=Tree` for small messages provides no benefit over Auto and risks suboptimal performance if gradient tensor sizes shift during training. The 0.8 GB/s advantage of Auto over forced Ring at peak bandwidth suggests that NCCL’s internal tuner performs optimisations — such as channel count adjustment and chunk size tuning — that are not accessible through the public `NCCL_ALGO` environment variable.

Never force NCCL_PROTO=LL for large gradient tensors. The 61.2% bandwidth penalty imposed by LL above 16 KB is severe and has no compensating benefit for large messages. LL’s low-latency flag mechanism is only advantageous in the regime below 16 KB where coordination latency dominates. For gradient tensors larger than 16 KB — which includes virtually all weight matrices in modern neural networks — LL128 or Auto should be preferred. Even LL128 imposes a 26.1% penalty at large message sizes, making Auto the only recommended choice for production workloads unless latency-sensitivity in the sub-16 KB regime is a priority.

Gradient bucketing thresholds matter. Distributed training frameworks such as PyTorch DDP accumulate gradients into buckets before calling AllReduce, with the default bucket size of 25 MB. Our results show that AllReduce performance improves significantly above 1 MB and continues climbing to the hardware ceiling at 8 GB. Bucket sizes well above 1 MB are strongly justified by these results, as they ensure AllReduce operates in the bandwidth-dominated regime where NVSwitch utilisation is high.

FSDP and ZeRO users should be aware of the AllGather anomaly at 32 MB. The sharp performance dip in AllGather at 32 MB is directly relevant to FSDP workloads, where AllGather is called to reconstruct full model layers before each forward pass. Transformer attention projection matrices — commonly sized at $d_{\text{model}} \times d_{\text{model}}$ — frequently fall near 32 MB for models with hidden dimensions around 4,096. Users experiencing unexpectedly low FSDP throughput on A100 NVSwitch hardware should investigate whether their layer sizes coincide with this anomaly boundary.

5.2 The Ring–Tree Crossover in Context

The observed Ring–Tree crossover window of 4–8 KB is narrow relative to the full message size range of 8 bytes to 8 GB. In practice, the gradient tensors that fall in this range include bias vectors and small projection matrices common in attention mechanisms. While the bandwidth advantage of Tree in this window is modest — approximately 7% at 4 KB — the advantage compounds across thousands of AllReduce calls per training step. For latency-sensitive pipelines where small gradient tensors are communicated frequently, NCCL Auto correctly exploits this window without requiring manual configuration.

The Tree performance anomaly at 64 MB is more consequential. A user who forces `NCCL_ALGO=Tree` believing it to be beneficial for medium-sized tensors would observe a 26.7% bandwidth drop at 64 MB. Auto avoids this entirely by

detecting the anomaly and routing to Ring at this message size, which is one reason Auto achieves a marginally higher peak than forced Ring.

5.3 Limitations

This study has several limitations that bound the generalisability of the findings.

Single-node only. All experiments ran within a single server. Multi-node AllReduce over InfiniBand or Ethernet introduces a network layer with fundamentally different latency and bandwidth characteristics. Algorithm and protocol crossover thresholds on multi-node configurations are expected to differ significantly from the intra-node results reported here.

Shared cloud infrastructure. Experiments ran on Vast.ai cloud instances rather than dedicated bare-metal hardware. While GPU model and interconnect type were confirmed via NCCL topology inspection, shared infrastructure introduces the possibility of host-level variability in CPU performance and PCIe contention. The findings are directionally robust but absolute bandwidth numbers should be interpreted as representative rather than definitive for a specific hardware configuration.

Single NCCL version. All experiments used NCCL 2.30.4. Algorithm selection thresholds and protocol crossover points are implementation-specific and may shift across NCCL versions as NVIDIA updates internal tuning tables.

AllReduce only for algorithm comparison. The Ring–Tree comparison was conducted only for AllReduce. AllGather and ReduceScatter may exhibit different crossover characteristics, as their internal implementations differ from AllReduce despite sharing the same underlying communication pattern.

6 Conclusion

Collective communication is the hidden tax on distributed training. Every gradient synchronisation call is a pause in useful compute, and the efficiency of that pause depends on decisions made deep inside NCCL that most practitioners never examine. This paper made those decisions visible.

On $8\times$ A100 SXM4 NVSwitch, the Ring–Tree algorithm choice is not a binary preference for one or the other but a function of message size with a precise crossover at 4–8 KB. Below this window, Tree’s logarithmic hop count reduces coordination latency. Above it, Ring’s full bandwidth utilisation dominates. The window is narrow but real, and Auto selection exploits it correctly — while also avoiding the undocumented Tree performance collapse at 64 MB that would penalise any user who forced Tree unconditionally for medium-sized tensors.

The protocol findings carry a stronger practical warning. A 61.2% bandwidth penalty for LL above 16 KB is not a marginal inefficiency — it is the difference between saturating the NVSwitch fabric and leaving most of it idle. Yet `NCCL_PROTO=LL` remains a configuration option that engineers occasionally set, either following outdated guidance or under the mistaken belief that low-latency protocol always means faster training. It does not, except for a narrow

regime that Auto already handles correctly.

The AllGather anomaly at 32MB is perhaps the most operationally consequential finding for practitioners who have not yet encountered it. FSDP workloads that happen to land transformer layers near this boundary will see unexpectedly degraded throughput with no obvious explanation in standard profiling output. Knowing the boundary exists is the first step to avoiding it.

The consistent lesson across all four experiments is the same: NCCL Auto selection is not a default to override — it is the result of careful hardware-specific tuning that manual configuration cannot easily replicate. The 0.8 GB/s advantage of Auto over forced Ring at peak bandwidth is a small number that represents something larger: an optimiser that knows things about channel counts and chunk sizes that are not exposed through environment variables.

All benchmark data, plotting scripts, and reproduction instructions are available at:

<https://github.com/michael-olagbemiro/nccl-benchmarks>

Acknowledgements

Experiments were conducted on GPU infrastructure provided by Vast.ai. The author thanks the open-source nccl-tests project maintained by NVIDIA for providing the benchmarking framework used in this study.

References

- [1] NVIDIA Corporation. *NCCL: NVIDIA Collective Communications Library*. <https://github.com/NVIDIA/nccl>, 2023.
- [2] NVIDIA Corporation. *nccl-tests: Tests for NCCL*. <https://github.com/NVIDIA/nccl-tests>, 2023.
- [3] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. In *Proceedings of SC20*, 2020.
- [4] M. Zhao, J. Gu, Z. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Sheng, A. Bhosale, T. Tian, B. Puri, M. Schroeder, G. Culatana, P. Balioglu, and T. Chen. *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. *Proceedings of the VLDB Endowment*, 16(12), 2023.
- [5] S. Sergeev and M. Del Balso. *Horovod: fast and easy distributed deep learning in TensorFlow*. arXiv:1802.05799, 2018.